



EXERCICE 3 · INTÉGRATION BONUS

Brancher le front sur l'API

Candidat : Sarah Brahimi · Sans date imposée · Saint-Laurent IT Solutions

Durée estimée : 2 à 3 heures. Prérequis : les exercices 1 et 2 terminés. Tu repars de tes deux rendus précédents.

Exercice bonus

Cet exercice est **facultatif**. Ne t'y attaque que si les exercices 1 et 2 sont terminés et rendus : ce sont eux qui comptent. Il n'y a pas de date imposée ici : prends le temps qu'il te faut, et préviens-nous simplement si tu décides de le faire.

Contexte

Tu as réparé un front. Tu as construit une API. Ce sont deux moitiés d'un même produit qui ne se sont jamais parlé. Or c'est précisément là que se passent la plupart des vrais problèmes : chacun fonctionne parfaitement isolé, et rien ne marche une fois branché.

Attention, un point important : l'application de l'exercice 1 gère des *tâches*, ton API de l'exercice 2 gère des *signalements de bugs*. Ce ne sont pas les mêmes données. C'est volontaire. Une partie du travail consiste à **adapter le front au modèle réel de l'API**, exactement ce qui arrive quand on branche une interface sur une API qui existait déjà et qu'on ne peut pas modifier.

Le travail demandé

Reprends ton application React et remplace les données en dur par des appels à ton API.

Action dans l'interface	Appel à l'API
Afficher la liste au chargement	GET /bugs
Ajouter un signalement	POST /bugs
Changer le statut d'un signalement	PATCH /bugs/:id
Supprimer un signalement	DELETE /bugs/:id

La case à cocher « terminée » n'a plus de sens : un signalement a trois statuts, pas deux états. À toi de choisir comment le représenter à l'écran, et de justifier ton choix dans le README.

Les trois points qui comptent vraiment

1. L'état de chargement

Entre le moment où la page s'ouvre et celui où les données arrivent, il se passe du temps. L'utilisateur doit voir qu'il se passe quelque chose, pas une page vide qui laisse croire qu'il n'y a rien.

2. L'état d'erreur

Éteins ton serveur, puis recharge la page. L'application doit afficher un message compréhensible, et surtout ne pas se figer sur un écran blanc. C'est le test le plus important de cet exercice : c'est ce que voit un client quand l'hébergeur a un incident.

3. L'adresse de l'API

Elle ne doit **jamais** être écrite en dur dans le code. Elle vit dans une variable d'environnement, avec un `.env.example` qui documente la clé. En local elle pointe sur `localhost`, en production ailleurs, et c'est la première chose qui casse un déploiement quand on l'a oubliée.

```
# .env.example, cote front (Vite)
VITE_API_URL=http://localhost:3000
```

Tu vas rencontrer CORS

Ton navigateur va très probablement refuser d'appeler ton API et afficher une erreur mentionnant CORS ou Access-Control-Allow-Origin. **C'est normal et c'est voulu.** Ce n'est pas un bug de ton code : c'est une protection du navigateur.

Cherche ce qu'est CORS, comprends pourquoi ça existe, corrige-le côté serveur, et **explique-le dans ton README avec tes propres mots**. Cette explication nous intéresse davantage que la correction elle-même.

Une question à laquelle répondre par écrit

À la fin de ton README, réponds à cette question en quelques lignes :

« Que se passe-t-il si l'utilisateur clique deux fois très vite sur le bouton Supprimer d'un même signalement ? Et qu'est-ce que tu ferais pour l'éviter ? »

Tu n'es pas obligée de coder la solution. Nous voulons voir si tu sais anticiper ce genre de situation : c'est ce qui distingue un développeur d'un assembleur de tutoriels.

Ce qui n'est pas demandé

Pas d'authentification, pas de pagination, pas de mise en ligne, pas de refonte graphique. Une interface simple qui parle correctement à ton API, et qui se comporte dignement quand l'API ne répond pas.

Consignes communes à tous les exercices

Elles comptent dans l'évaluation autant que le code lui-même.

- Un dépôt **GitHub public** par exercice, ou un dépôt avec un dossier par exercice.
- Tu travailles **sur une branche**, jamais directement sur `main`.
- **Commits atomiques** : un commit = un changement qui se raconte en une phrase. Évite les messages « fix », « update » ou « modif ».
- Quand tu as fini, tu ouvres une **Pull Request** de ta branche vers ton `main`, et tu décris dedans ce que tu as fait et pourquoi. **Tu ne la merges pas** : elle sera relue comme nous relisons celles de nos clients.
- Un fichier `README.md` à la racine : comment lancer le projet, ce que tu as fait, ce que tu n'as pas réussi, et le temps que tu y as passé.
- Tu peux utiliser l'IA. Mais tu dois pouvoir expliquer **chaque ligne** que tu rends : des questions précises seront posées dessus.

Si tu bloques

Si un point te bloque plus d'une heure, **note-le dans le README et passe à la suite**. Un exercice à moitié fini mais bien expliqué vaut mieux qu'un exercice fini que tu ne sais pas justifier. Et tu peux nous écrire à tout moment : savoir demander de l'aide au bon moment fait partie du métier.